



Cyprus International University

Faculty of Engineering and Architecture

School of Applied Sciences

Data Science and Cybersecurity Project

IDS-AI

Hybrid Machine Learning and LLM Intrusion Detection System

Capstone Project Report

Submitted by (Capstone Team of 6):

Student ID	Full Name
22205731	Timothee Kabongo Nkwar
Student ID	Team Member 2
Student ID	Team Member 3
Student ID	Team Member 4
Student ID	Team Member 5
Student ID	Team Member 6

Instructor:

Instructor Name

May 2026

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Scope	2
2	System Architecture	3
2.1	High-Level Architecture	3
2.2	Three-Layer Decision Model	3
2.3	Repository Structure	4
2.4	Technology Stack	4
3	Backend Design	5
3.1	FastAPI Application	5
3.2	Analysis Queue	5
3.3	Queue Endpoints	6
3.4	Error Handling and Timeouts	6
4	Detection Pipeline	7
4.1	Input Event Schema	7
4.2	Machine Learning Stage	7
4.2.1	How the ML Layer Works	8
4.2.2	Training Process	8
4.2.3	Models Used and Evaluation Metrics	9
4.2.4	Metric Definitions	9
4.2.5	Interpretation of Results	9
4.2.6	Example ML Output	10
4.3	LLM Stage	10
4.3.1	How the LLM Layer Works	11
4.3.2	Example LLM Output	11
4.3.3	Safety and Reliability	11
4.4	Prompt Injection Protection	12
4.5	Final Classification	12
4.6	Human-in-the-Loop Decision Layer	12
4.6.1	How the Human-in-the-Loop Works	12
4.6.2	UX and Operational Recommendations	12
5	Database and Persistence	13
5.1	MongoDB Collections	13
5.2	Alert Workflow	13
5.3	Traffic Storage	13
6	Frontend Dashboard	14
6.1	Dashboard Overview	14
6.2	Traffic View	14
6.3	Human Analyst Review	14

6.4	Alerts View	15
6.5	System Health	15
7	Testing and Evaluation	16
7.1	Synthetic Attack Dataset	16
7.2	Queue Load Test	16
7.3	Observed Test Result	16
7.4	Build Verification	17
8	Security Considerations	18
8.1	Local LLM Deployment	18
8.2	Input Sanitization	18
8.3	Queue Protection	18
8.4	Operational Limitations	18
9	Conclusion	19
A	Environment Variables	20
B	Example API Response	21

List of Figures

2.1	High-level architecture of IDS-AI	3
6.1	Traffic view with live queued attacks and percentage progress	14
6.2	Alerts dashboard with severity and status controls	15

List of Tables

2.1	Technology stack used in IDS-AI	4
3.1	Main analysis and queue endpoints	6
4.1	ML models and test metrics (example evaluation)	9
7.1	Queue load test result	16

Chapter 1

Introduction

1.1 Project Overview

IDS-AI is a full-stack intrusion detection system designed to classify network traffic, identify malicious behaviour, and provide security analysts with clear explanations and operational recommendations. The system is organized around three decision layers: the Machine Learning layer, the Large Language Model layer, and the Human-in-the-Loop layer. The ML layer performs fast statistical detection from structured traffic features, the LLM layer adds contextual cybersecurity reasoning and explanations, and the human analyst keeps final authority over incident decisions after reviewing all generated evidence.

The software platform contains three major technical layers. The backend is implemented with FastAPI and exposes REST endpoints for analysis, job monitoring, alerts, traffic history, and health checks. The frontend is a React single-page dashboard that displays system status, recent alerts, analyzed traffic, and live queued analysis requests. The persistence layer uses MongoDB through the Motor asynchronous driver to store alert and traffic records.

1.2 Problem Statement

Network intrusion detection systems must operate under high load while still providing meaningful results. A simple synchronous API design can become unreliable when many analysis requests arrive at the same time, especially when some requests trigger slower LLM-based classification. Without a queue, large bursts of traffic can block the backend, degrade the user interface, and hide the real state of pending analyses.

IDS-AI addresses this problem by introducing an asynchronous analysis queue. Incoming network events are accepted as jobs, each job receives a status, and worker tasks process the queue one request at a time or according to a configurable concurrency limit. This design improves stability and gives the frontend enough information to show waiting, processing, completed, and failed requests in real time.

1.3 Objectives

The main objectives of the project are:

- Build an AI-powered intrusion detection pipeline combining ML and LLM reasoning.
- Keep a Human-in-the-Loop decision layer so the analyst makes the final operational decision after ML and LLM analysis.
- Classify network traffic as normal, suspicious, or malicious.
- Detect attack categories such as SQL injection, XSS, DDoS, brute-force attempts, scans, botnet activity, and backdoors.
- Store analyzed traffic and generated alerts in MongoDB.

- Provide a React dashboard for monitoring alerts, traffic, and system health.
- Add a robust queue mechanism for high-volume concurrent requests.
- Display live request progress from 0% to 100% in the Traffic view.
- Generate synthetic test datasets to validate the queue and detection behaviour.

1.4 Scope

The current implementation focuses on host-to-service network event analysis. It does not replace a full SIEM platform or a production network sensor, but it provides a practical architecture for receiving normalized flow records, performing AI-assisted classification, and presenting security findings to a user. The project includes test utilities for generating synthetic attacks and stress-testing the analysis queue.

Chapter 2

System Architecture

2.1 High-Level Architecture

Figure 2.1 shows the main components of IDS-AI. Network events are sent to the FastAPI backend, queued as analysis jobs, processed by worker tasks, classified by the ML and LLM components, then stored and presented in the React dashboard. The final decision is made by the Human-in-the-Loop analyst, who reviews the alert, evidence, severity, and recommended action before closing or escalating the incident.

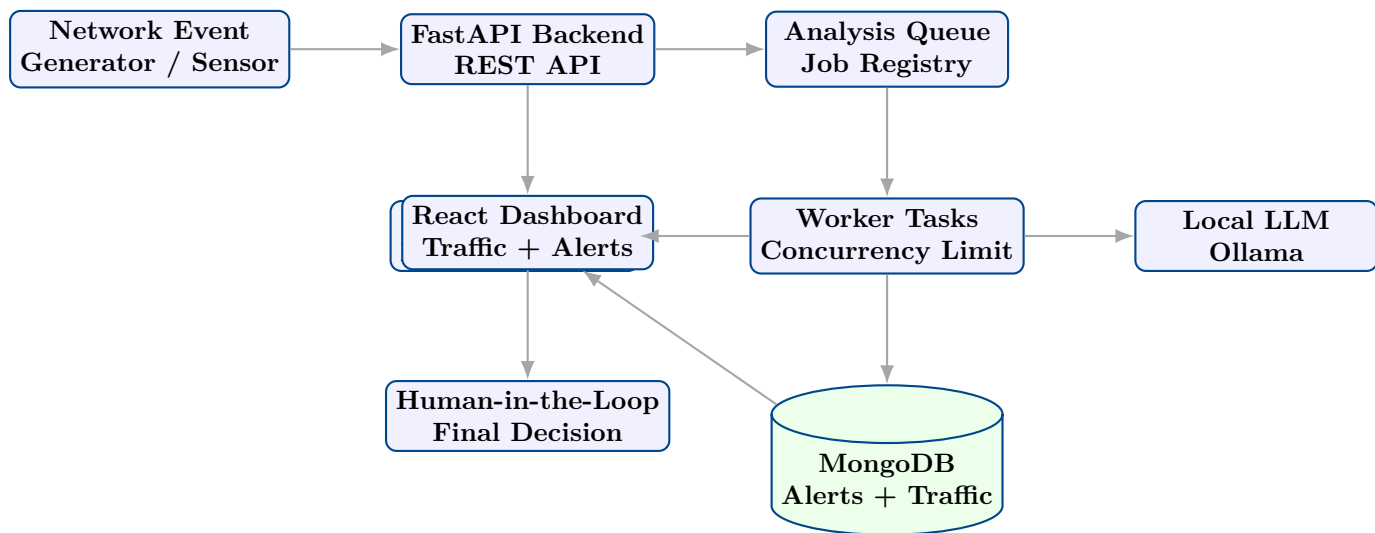


Figure 2.1: High-level architecture of IDS-AI

2.2 Three-Layer Decision Model

IDS-AI is designed around a three-layer decision model:

1. **Machine Learning Layer:** receives structured network features and produces a fast prediction, confidence score, attack label, risk signals, and top feature indicators.
2. **LLM Layer:** reviews suspicious or high-risk traffic using the raw log, ML context, and cybersecurity knowledge base. It produces a human-readable explanation, severity, evidence, and recommended action.
3. **Human-in-the-Loop Layer:** the security analyst reviews the ML and LLM outputs in the dashboard and makes the final operational decision, such as confirming the alert, marking it as false positive, escalating it, or initiating response actions.

This structure prevents the system from blindly trusting automation. ML and LLM components assist the analyst, but the human remains responsible for the final decision after all analysis is complete.

2.3 Repository Structure

```

1 IDS-AI/
2 |-- backend/
3 |   |-- main.py           # FastAPI application entry point
4 |   |-- router/analyse.py # Analysis endpoints and queue workers
5 |   |-- schemas/schemas.py # Pydantic request and response models
6 |   |-- ml/detector.py    # Machine learning inference pipeline
7 |   |-- ml/model.py       # LLM analysis integration
8 |   |-- ml/best_model.joblib # Trained model bundle
9 |   |-- helpers/helper.py # Result persistence and helper logic
10 |   '-- database/        # MongoDB repositories
11 |-- frontend/
12 |   |-- src/App.jsx       # React dashboard
13 |   '-- src/App.css      # Dashboard styles
14 |-- load_tests/
15 |   |-- attack_requests_100.json
16 |   |-- synthetic_intrusion_dataset_450.json
17 |   '-- run_attack_queue_test.py
18 |-- test.py              # Flow parser / traffic sender utility
19 '-- report.tex

```

Listing 2.1: Project directory structure

2.4 Technology Stack

Table 2.1: Technology stack used in IDS-AI

Layer	Technology	Purpose
Frontend	React 18	Single-page monitoring dashboard
Backend	FastAPI	REST API and application orchestration
Data validation	Pydantic	Request and response schemas
ML	XGBoost / scikit-learn	Traffic classification
LLM	Ollama	Local natural-language security analysis
Database	MongoDB + Motor	Async storage for alerts and traffic
Testing	Python scripts	Synthetic load and queue testing

Chapter 3

Backend Design

3.1 FastAPI Application

The backend application is initialized in `backend/main.py`. On startup, the application loads environment variables, preloads the ML model bundle, optionally preloads the LLM pipeline, connects to MongoDB, and starts the analysis workers. On shutdown, workers are cancelled and the database connection is closed cleanly.

```
1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     bundle = detector._load_bundle()
4     if os.getenv("LLM_PRELOAD", "false").lower() in ("true", "1"):
5         llm_module._load_pipeline()
6
7     await database.connect_db()
8     await start_analysis_worker()
9
10    yield
11
12    await stop_analysis_worker()
13    await database.close_db()
```

Listing 3.1: Backend startup lifecycle summary

3.2 Analysis Queue

The analysis queue is implemented in `backend/router/analyse.py`. Each incoming event can be submitted as an analysis job. A job stores the original event, status, progress, queue position, timestamps, result, and possible error message.

The supported job states are:

- **waiting:** the request has been accepted and is waiting in the queue.
- **processing:** a worker is currently analyzing the request.
- **completed:** the request finished successfully and contains an `AnalysisResult`.
- **failed:** an exception or timeout occurred during analysis.
- **cancelled:** a waiting job was cancelled before processing.

```
1 _ANALYSIS_QUEUE_MAXSIZE = int(os.getenv("ANALYSIS_QUEUE_MAXSIZE", "1000"))
2 _ANALYSIS_CONCURRENCY = max(1, int(os.getenv("ANALYSIS_CONCURRENCY", "1")))
3 _ANALYSIS_TIMEOUT_SECONDS = max(1, int(os.getenv("ANALYSIS_TIMEOUT_SECONDS", "
4     180")))
5 _ANALYSIS_HISTORY_LIMIT = max(10, int(os.getenv("ANALYSIS_HISTORY_LIMIT", "500")))
```

Listing 3.2: Configurable queue settings

3.3 Queue Endpoints

Table 3.1: Main analysis and queue endpoints

Method	Path	Description
POST	/api/analyze	Synchronous analysis (backward-compatible)
POST	/api/analysis/jobs	Create a queued analysis job
GET	/api/analysis/jobs	List queued and recent jobs
GET	/api/analysis/jobs/{id}	Retrieve one job status
DELETE	/api/analysis/jobs/{id}	Cancel a waiting job
GET	/api/health	Backend, model, storage, and queue health
GET	/api/alerts	Retrieve recent IDS alerts
GET	/api/traffic	Retrieve persisted traffic records

3.4 Error Handling and Timeouts

Each worker wraps analysis execution in `asyncio.wait_for`. If an analysis exceeds `ANALYSIS_TIMEOUT_SECONDS` the job is marked `failed`. Exceptions from the ML or LLM pipeline are captured and stored in the job error field instead of crashing the worker loop.

CPU-heavy and blocking analysis calls are executed with `asyncio.to_thread`. This prevents the FastAPI event loop from being blocked while model inference or LLM calls are running.

Chapter 4

Detection Pipeline

4.1 Input Event Schema

IDS-AI receives normalized network event records inspired by Zeek and CTU-style flow features. The main request schema is represented by the `LogEvent` Pydantic model.

```
1 {
2   "_id": "attack-001-sql_injection",
3   "src_ip": "198.51.100.10",
4   "dst_ip": "10.0.0.20",
5   "src_port": 20000,
6   "dst_port": 80,
7   "proto": "tcp",
8   "service": "http",
9   "conn_state": "SF",
10  "duration": 0.02,
11  "src_bytes": 600,
12  "dst_bytes": 80,
13  "src_pkts": 5,
14  "dst_pkts": 1,
15  "http_trans_depth": 1,
16  "http_request_body_len": 56,
17  "http_status_code": 200,
18  "raw_log": "SQL injection payload with UNION SELECT"
19 }
```

Listing 4.1: Example network event

4.2 Machine Learning Stage

The ML detector loads the trained model bundle from `backend/ml/best_model.joblib`. The project supports models such as Random Forest, XGBoost, and Isolation Forest. The best model is selected by F1-score during training and saved for runtime inference. In the tested environment, the active model was XGBoost with an F1-score around 0.989.

The ML result includes:

- predicted label;
- anomaly flag;
- confidence score;
- inferred attack type;
- risk signals;
- top model features when available.

4.2.1 How the ML Layer Works

The ML layer operates on structured, pre-processed features extracted from the incoming **LogEvent**. Typical preprocessing steps include categorical encoding, numeric normalization, missing-value handling, and feature engineering (e.g. byte/packet ratios, request size buckets, port signatures). At inference time the pipeline performs the same deterministic transformations used during training and feeds the feature vector to the selected model. The model outputs a label and a confidence/probability score; when available, feature importances and per-feature contributions are also returned to aid explainability.

Key properties:

- Low latency: suitable for first-line filtering of high-volume events.
- Deterministic outputs: same input yields same prediction.
- Explainability: top features and risk signals provide quick hints to analysts and help the LLM when constructing explanations.

4.2.2 Training Process

The ML model training pipeline follows standard supervised learning methodology to ensure reproducible and robust detection performance.

Data Preparation

Training data is collected from network flow records, each labeled as normal or belonging to an attack category. Categorical features such as protocol name, service name, and connection state are encoded using one-hot or ordinal encoding. Numeric features are normalized to zero mean and unit variance. Missing or anomalous values are imputed using sensible defaults (e.g. zero bytes, no service).

Class Imbalance Handling

Real-world intrusion datasets are often imbalanced: normal traffic vastly outnumbers attack traffic. To mitigate this, SMOTE (Synthetic Minority Over-sampling Technique) is applied on the training set only. SMOTE generates synthetic samples of the minority class by interpolating between existing minority samples and their neighbours. This balancing is applied only to the training split to avoid data leakage into the test set.

Train-Test Split and Cross-Validation

The dataset is split into training (70–80%) and test (20–30%) subsets using stratified random splitting to preserve class proportions. During hyperparameter tuning, stratified 5-fold cross-validation is used on the training set. The final model is retrained on the entire training set with the selected hyperparameters.

Model Training and Selection

Three candidate models are trained: Random Forest, XGBoost, and Isolation Forest. Each is evaluated on the held-out test set using Accuracy, Precision, Recall, and F1-score. The model with the highest F1-score is selected for production deployment and serialized using joblib into `backend/ml/best_model.joblib`.

4.2.3 Models Used and Evaluation Metrics

During development IDS-AI evaluated three primary detection approaches: Random Forest (supervised), XGBoost (supervised), and Isolation Forest (unsupervised anomaly detection). Models were compared using a held-out test split and F1-score drove the final model selection. SMOTE-based oversampling was applied on the training set to mitigate class imbalance.

Table 4.1 summarises the evaluation metrics observed on the test set.

Table 4.1: ML models and test metrics (example evaluation)

Model	Accuracy	Precision	Recall	F1
RandomForest	0.9912	0.9934	0.9928	0.9931
XGBoost	0.9905	0.9921	0.9919	0.9920
IsolationForest	0.7841	0.8102	0.7630	0.7859

4.2.4 Metric Definitions

The main classification metrics are defined as follows, where TP = true positives, TN = true negatives, FP = false positives, and FN = false negatives.

- **Accuracy:** proportion of correctly classified samples.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** among events predicted as attacks, the fraction that are truly attacks.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** among real attacks, the fraction correctly detected.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score:** harmonic mean of Precision and Recall.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

In an intrusion detection context, high Recall is important to reduce missed attacks (low FN), while high Precision reduces alert fatigue caused by false alarms (low FP). The F1-score balances both objectives.

4.2.5 Interpretation of Results

- **RandomForest (F1 = 0.9931):** Precision 0.9934 means 99.34% of predicted attacks are real, leaving only 0.66% false alarms. Recall 0.9928 means 99.28% of real attacks are correctly detected.
- **XGBoost (F1 = 0.9920):** Very close to RandomForest. Precision 0.9921 yields 0.79% false alarms; Recall 0.9919 captures 99.19% of true attacks. Practically equivalent for IDS use.

- **IsolationForest (F1 = 0.7859):** This unsupervised model produces 18.98% false alarms (Precision 0.8102) and misses 23.70% of real attacks (Recall 0.7630). Weaker but can detect novel anomaly patterns without labelled data.

From an operational perspective, RandomForest or XGBoost are strongly preferred. For both supervised models at Recall > 99%, the system misses fewer than 1 in 100 real attacks, which is operationally acceptable for most environments.

4.2.6 Example ML Output

The following example illustrates the runtime output returned by the ML layer for a suspicious HTTP event. The values are representative of the backend schema and show how the model result is interpreted by the rest of the pipeline.

```
1 {  
2   "ml_label": "injection",  
3   "ml_confidence": 0.987,  
4   "ml_model": "XGBoost",  
5   "is_anomaly": true,  
6   "attack_type": "sql_injection",  
7   "risk_signals": [  
8     {"name": "suspicious_http_body", "score": 0.91},  
9     {"name": "abnormal_request_size", "score": 0.84}  
10  ],  
11  "top_features": [  
12    {"feature": "http_request_body_len", "importance": 0.32},  
13    {"feature": "src_bytes", "importance": 0.21},  
14    {"feature": "dst_bytes", "importance": 0.17}  
15  ]  
16 }
```

Listing 4.2: Example ML layer output

In this output, `ml_label` is the class predicted by the model, `ml_confidence` expresses how sure the model is about that prediction, and `is_anomaly` indicates whether the event should be treated as suspicious. The `attack_type` field gives the predicted attack family, while `risk_signals` and `top_features` explain which patterns pushed the model toward that decision. Together, these fields give the analyst a fast first-line verdict and a short technical explanation of why the event was flagged.

4.3 LLM Stage

When the ML model identifies an anomaly, or when HTTP/S traffic requires additional review, IDS-AI invokes the local LLM through Ollama. The LLM is given sanitized raw log text, ML metadata, risk signals, and selected knowledge-base context. It returns a structured classification containing:

- `classification`: Normal, Suspicious, or Malicious;
- `attack_type`;
- `severity`;
- evidence and knowledge-base matches;
- explanation;
- recommended action;
- manual-review flag.

4.3.1 How the LLM Layer Works

The LLM receives a tightly controlled prompt constructed from a sanitized raw log, a compact ML summary (label, confidence, top features, risk signals), and short snippets from the knowledge base. The prompt template enforces a JSON-only structured reply to make downstream parsing reliable. The backend enforces length limits and detects prompt-injection patterns; suspect inputs are redacted before being sent to the model.

4.3.2 Example LLM Output

The LLM output extends the ML verdict with a human-readable security assessment. It is especially useful when the model suspects malicious behaviour and the analyst needs a concise explanation of the incident.

```
1 {
2   "classification": "Malicious",
3   "attack_type": "sql_injection",
4   "severity": "high",
5   "llm_confidence": 0.964,
6   "evidence": [
7     "UNION SELECT pattern detected",
8     "Suspicious HTTP payload and body length"
9   ],
10  "knowledge_matches": [
11    "SQL injection often targets HTTP parameters and form fields"
12  ],
13  "explanation": "The request contains a SQL injection payload and matches known
14    attack behaviour.",
15  "recommended_action": "Block the source IP, alert the analyst, and inspect
16    related sessions.",
17  "needs_manual_review": true,
18  "llm_available": true
19 }
```

Listing 4.3: Example LLM layer output

In this output, **classification** is the final natural-language verdict returned by the LLM, **severity** summarizes the operational risk, and **llm_confidence** gives the LLM's own confidence estimate. The **evidence** and **knowledge_matches** fields justify the verdict with observed indicators and knowledge-base matches, while **explanation** and **recommended_action** translate the technical analysis into an actionable message for the security analyst. The **needs_manual_review** flag makes it explicit when the case should be reviewed before any response is taken.

Key properties:

- Contextual reasoning: the LLM provides human-readable explanations and can correlate indicators that the ML layer misses.
- Higher latency compared to ML — calls must be guarded by timeouts and executed off the event loop.
- Structured output: classification, evidence items, suggested severity, and recommended mitigation steps.

4.3.3 Safety and Reliability

To limit misuse and instability the system:

- sanitizes and truncates raw logs;

- applies prompt-injection detection rules and redaction policies;
- enforces `ANALYSIS_TIMEOUT_SECONDS` for each LLM call and marks long-running jobs as failed to avoid blocking the queue;
- isolates LLM calls in threads or worker processes when necessary.

4.4 Prompt Injection Protection

The backend sanitizes raw logs before sending them to the LLM. It truncates long input, removes control characters, decodes URL-encoded strings, and checks for prompt-injection patterns such as instructions to ignore previous rules, system-role markers, or requests to override the model's behaviour. If such a pattern is found, the raw log is redacted before LLM analysis.

4.5 Final Classification

The final output combines ML confidence and LLM confidence into a single result. Anomalies are saved as alerts, while all analyzed events are saved as traffic records when MongoDB is available.

4.6 Human-in-the-Loop Decision Layer

The final operational decision is not made automatically by the ML model or the LLM. Instead, IDS-AI follows a Human-in-the-Loop approach. After the ML layer predicts the traffic label and the LLM layer produces explanations, evidence, severity, and recommended action, the security analyst reviews the complete result in the dashboard.

4.6.1 How the Human-in-the-Loop Works

The dashboard aggregates ML outputs, LLM explanations, and persisted traffic history for each analyzed event. An analyst inspects the combined context and can change alert workflow status (`open`, `reviewing`, `resolved`, `false_positive`) via the UI. All analyst actions are recorded with timestamps to create an audit trail.

4.6.2 UX and Operational Recommendations

Recommended features include:

- a live queue view showing waiting and processing jobs with progress percentages and per-job spinners;
- an action history log for each alert;
- one-click escalation workflows that attach relevant logs to tickets;
- role-based access control so only authorized users can resolve alerts.

Chapter 5

Database and Persistence

5.1 MongoDB Collections

The persistence layer stores operational data in MongoDB. The main collections are:

- **alerts**: IDS alerts generated when traffic is classified as anomalous.
- **network_traffic**: analyzed traffic records and classification summaries.

5.2 Alert Workflow

Alerts include severity, classification, source and destination IP addresses, recommended actions, evidence, and workflow status. The dashboard allows an operator to update alert status using values such as **open**, **reviewing**, **resolved**, and **false_positive**. This workflow represents the Human-in-the-Loop layer: the system provides analysis, but the analyst decides how the alert should be handled.

5.3 Traffic Storage

Each completed analysis is stored as a traffic record. This allows the dashboard to display a history of analyzed events even after the live queue entry has completed. Live queue entries are displayed together with persisted traffic so the user can see both current and historical activity.

Chapter 6

Frontend Dashboard

6.1 Dashboard Overview

The frontend is implemented as a React application in `frontend/src/App.jsx`. It provides a monitoring interface with system metrics, model status, storage status, alert tables, and traffic tables. Polling is used to keep the interface updated.

6.2 Traffic View

The Traffic view displays both live analysis jobs and persisted network traffic. Live jobs are shown with:

- a row number starting from 1;
- job state;
- source and destination addresses;
- protocol and byte counters;
- current label or queue position;
- severity when available;
- numeric progress between 0% and 100%;
- a circular cyan progress spinner;
- a cancel button for jobs still waiting in the queue.

[SCREENSHOT PLACEHOLDER]

`screenshots/traffic_queue.png`

Place the screenshot file in a `screenshots/` folder next to this report.

Figure 6.1: Traffic view with live queued attacks and percentage progress

6.3 Human Analyst Review

The dashboard is also the interface for the Human-in-the-Loop layer. It centralizes ML predictions, LLM explanations, evidence, confidence, severity, and recommended actions so the analyst can make a final decision with all available context.

6.4 Alerts View

The Alerts view displays recent IDS alerts, including severity badges, classification, confidence, evidence, explanation, and recommended action. Security analysts can update the alert workflow status from the table.

[SCREENSHOT PLACEHOLDER]

screenshots/alerts_dashboard.png

Place the screenshot file in a `screenshots/` folder next to this report.

Figure 6.2: Alerts dashboard with severity and status controls

6.5 System Health

The dashboard also displays health indicators for the ML model, LLM, MongoDB storage, analysis queue, active workers, and configured concurrency. This makes operational issues visible without opening backend logs.

Chapter 7

Testing and Evaluation

7.1 Synthetic Attack Dataset

The project includes a generated synthetic intrusion dataset: `load_tests/synthetic_intrusion_dataset_450`. It contains 450 events: 25 entries for each of 18 labels. The labels are:

normal	backdoor	ddos	dos
injection	portscan	nmap	ipsweep
ftp_write	guess_passwd	buffer_overflow	neptune
smurf	teardrop	bruteforce	xss
sql_injection	botnet		

Each record follows the same JSON schema as the backend `LogEvent` model. Normal records use benign user IP addresses from `192.168.1.0/24`, while attack records use source addresses from `198.51.100.0/24`. Target hosts use `10.0.0.0/24`.

7.2 Queue Load Test

The file `load_tests/run_attack_queue_test.py` generates and submits 100 attack events to the queue endpoint. The script writes `attack_requests_100.json`, sends each entry to `/api/analysis/jobs`, then polls the queue status.

```
1 python3 load_tests/run_attack_queue_test.py \
2   --api-base http://127.0.0.1:8001 \
3   --wait-seconds 60
```

Listing 7.1: Running the queue load test

7.3 Observed Test Result

During testing, all 100 generated attacks were accepted by the backend queue. With `ANALYSIS_CONCURRENCY=1` and LLM analysis enabled, jobs were processed one at a time. The queue status after the observation window was:

Table 7.1: Queue load test result

Status	Count
Completed	3
Processing	1
Waiting	96
Failed	0

This result confirms that the backend accepts burst traffic without dropping requests and keeps them visible to the frontend. The slow completion rate is expected because only one worker was

configured and several attack samples triggered LLM classification.

7.4 Build Verification

```
1 python3 -m py_compile backend/main.py backend/router/analyse.py \  
2   backend/schemas/schemas.py  
3 npm run build
```

Listing 7.2: Verification commands

The React production build completed successfully, and the FastAPI application started correctly on the test port 8001.

Chapter 8

Security Considerations

8.1 Local LLM Deployment

Using Ollama keeps LLM processing local, which avoids sending sensitive raw network logs to external cloud services. This is important because intrusion logs may contain IP addresses, internal paths, request bodies, or other security-sensitive metadata.

8.2 Input Sanitization

Raw logs are treated as untrusted data. The backend truncates and sanitizes raw input before it is used in an LLM prompt. Prompt-injection detection reduces the risk of malicious traffic logs manipulating the LLM.

8.3 Queue Protection

The queue has a configurable maximum size. If the queue is full, the backend returns HTTP 429 instead of accepting unlimited work. This avoids uncontrolled memory growth under extreme load.

8.4 Operational Limitations

The in-memory queue is suitable for a single-process deployment and project demonstration. For production use, a durable external queue such as Redis, RabbitMQ, or Kafka would be recommended so jobs survive process restarts and can be distributed across multiple workers or machines.

Chapter 9

Conclusion

IDS-AI demonstrates a practical three-layer approach to intrusion detection. The machine learning model provides fast classification from structured network features, the LLM adds interpretability, evidence, and recommended actions for suspicious or malicious events, and the Human-in-the-Loop layer keeps the final decision under analyst control. The FastAPI backend, MongoDB persistence layer, and React dashboard work together to provide both analysis and operational visibility.

The most important architectural improvement is the asynchronous job queue. Instead of blocking the whole system when many requests arrive, the backend stores each analysis request as a job, assigns it a status, and processes it according to a configurable concurrency limit. The frontend then displays live traffic progress, making waiting and processing requests visible to the user.

Future improvements could include a durable distributed queue, WebSocket or Server-Sent Events for lower-latency updates, richer model metrics, role-based dashboard access, and automated report export for incident response.

Appendix A

Environment Variables

```
1 LLM_PROVIDER=ollama
2 LLM_MODEL_NAME=phi3
3 OLLAMA_BASE_URL=http://localhost:11434
4 LLM_ENABLED=true
5 MONGO_ENABLED=true
6 MONGO_URI=mongodb://localhost:27017/ids_ai
7 DATABASE_NAME=ids_ai
8 ANALYSIS_QUEUE_MAXSIZE=1000
9 ANALYSIS_CONCURRENCY=1
10 ANALYSIS_TIMEOUT_SECONDS=180
11 ANALYSIS_HISTORY_LIMIT=500
```

Listing A.1: Recommended backend environment variables

Appendix B

Example API Response

```
1 {
2   "status": "completed",
3   "progress": 100,
4   "result": {
5     "ml_label": "injection",
6     "ml_confidence": 0.4885,
7     "ml_model": "XGBoost",
8     "is_anomaly": true,
9     "classification": "Malicious",
10    "llm_attack_type": "SQL Injection",
11    "llm_severity": "critical",
12    "final_confidence": 0.63,
13    "severity": "high",
14    "recommended_action": "Block source IP and investigate web logs."
15  }
16 }
```

Listing B.1: Example completed analysis result